Task for ASG

benefits of Auto Scaling Groups (ASG):

- Maintains **high availability** by automatically replacing unhealthy EC2 instances.
- Provides **elasticity** by scaling out during high traffic and scaling in during low traffic.
- Ensures **cost optimization** by running only the required number of instances.
- Improves **fault tolerance** by distributing instances across multiple Availability Zones.
- Supports **predictable performance** using scheduled scaling for known traffic patterns.
- Integrates with **CloudWatch alarms** to respond instantly to workload changes.
- Simplifies operations with **automated health checks, rebalancing, and lifecycle management**.

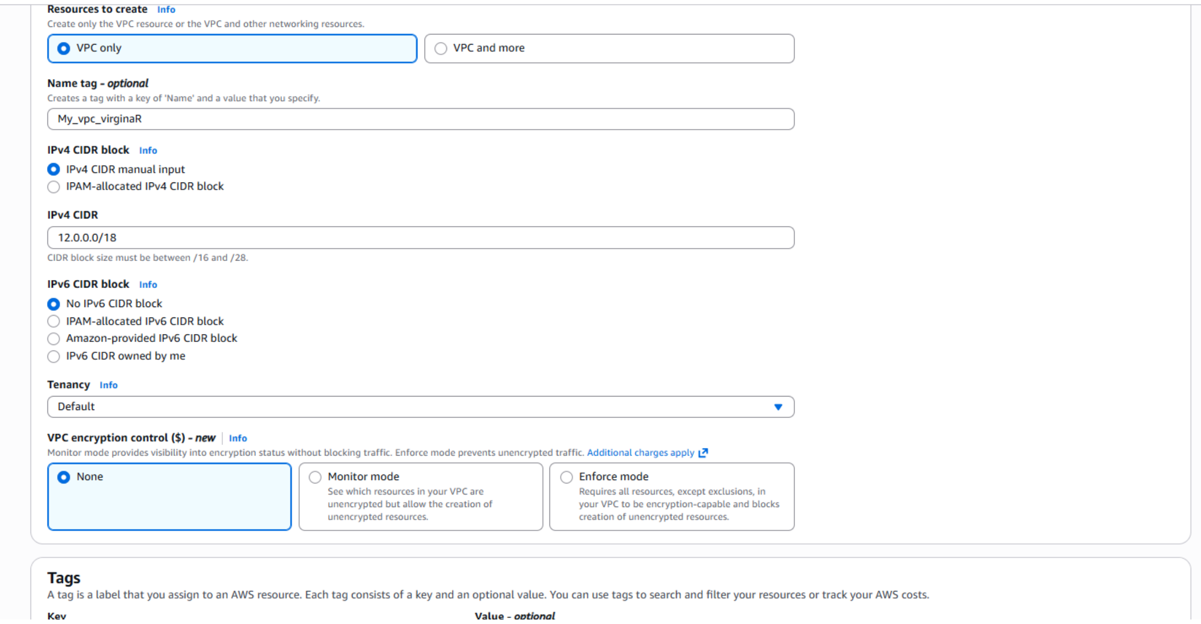
Tools we will be using:

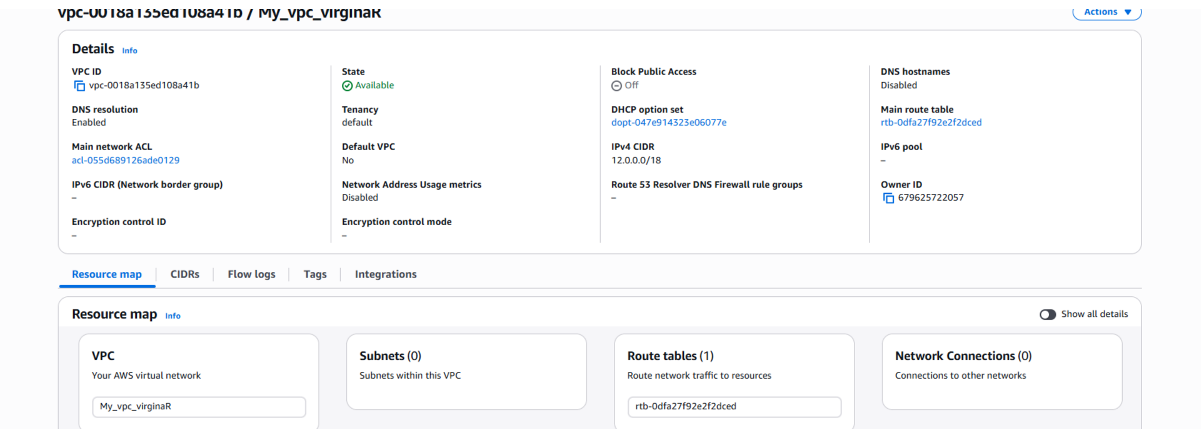
- **VPC** → Networking setup (VPC, subnets, route tables, IGW, NAT).
- **EC2** → Launch instances & deploy Apache.
- **S3** → Store ALB logs.
- **ALB** → Distribute traffic to EC2s.
- **CloudWatch** → Monitor CPU & Apache service.
- **CloudWatch Logs** → Store VPC Flow Logs.
- **ASG** → Auto-scale EC2 when CPU > 70%.
- **Launch Template** → Blueprint for auto-launched instances.
- **IAM** → Permissions for CloudWatch, ASG, logs.

1. Create one VPC in N. Virginia region.

Steps:

- **Go AWS then Go to N.virginia region console < navigate to vpc < create vpc**
- **Selected my own cidr number with 18 subnet musk**
- **Created vpc and it is shown in the images below.**

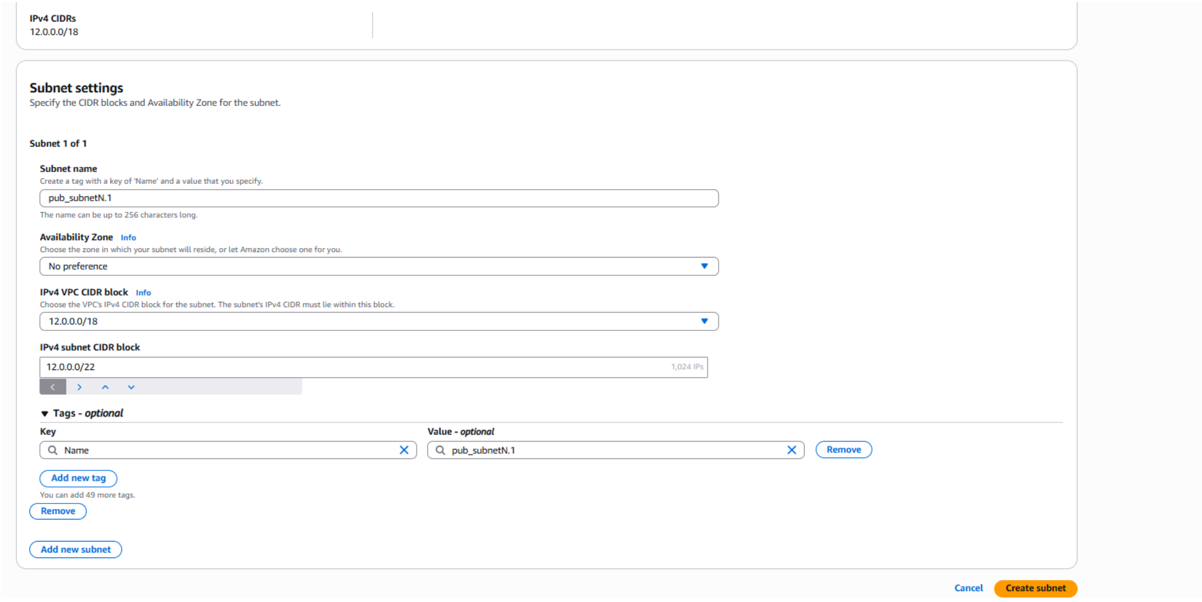


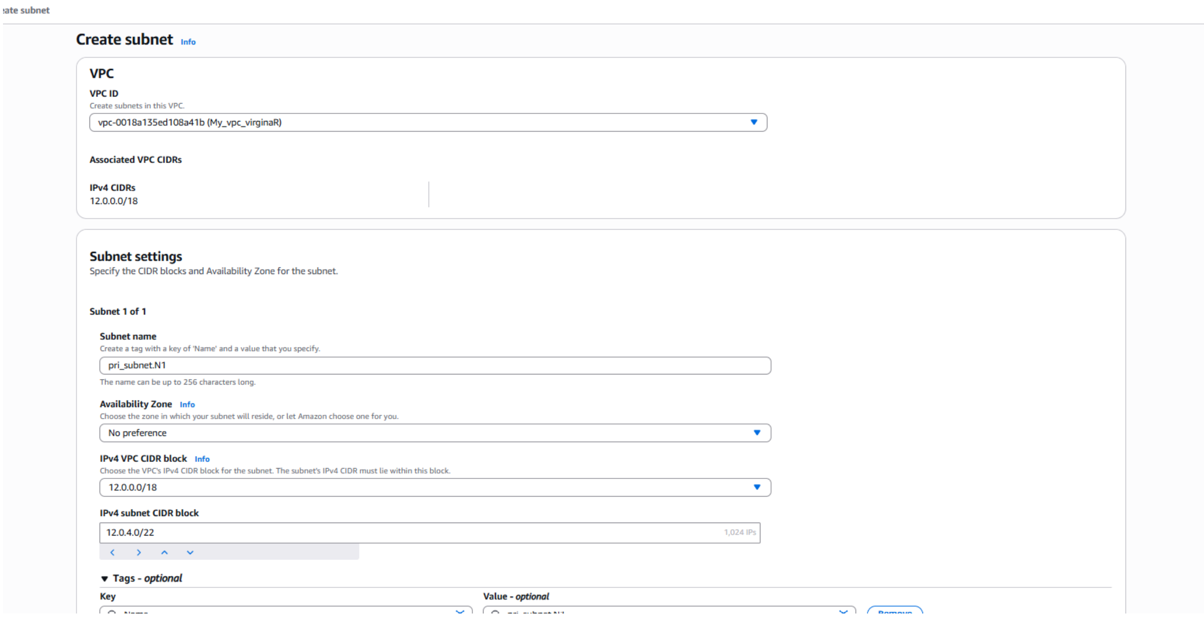


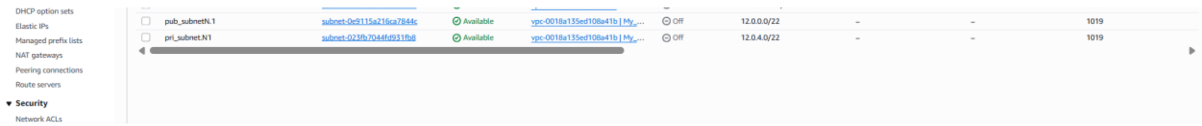
2. Create two subnets: one public subnet and one private subnet.

Steps:

- Created 2 subnets, go to aws console < go to subnets < go to create subnets
- Create public subnet < select our vpc < select cidr range
- Create private subnet < select vpc < select cidr range for private subnet
- Create subnet both subnets are created based the steps above.



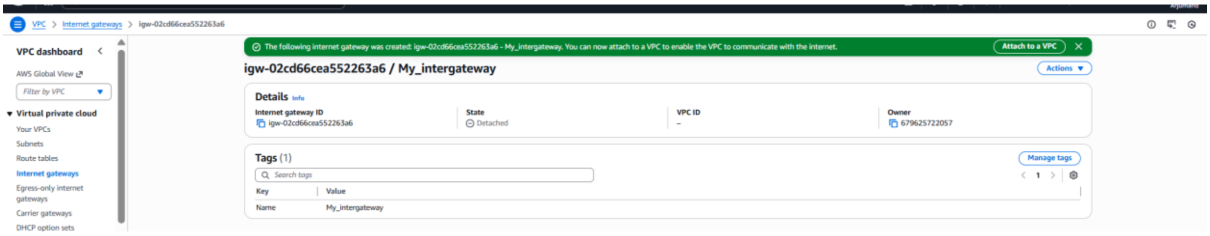


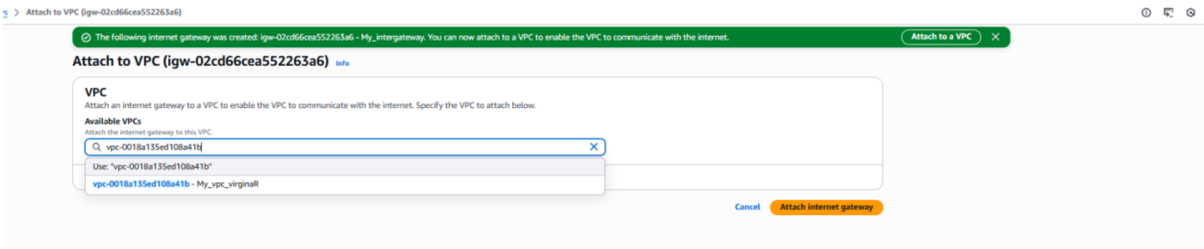


3. Attach an IGW to the VPC.

Steps:

- ➢ Create a inter gateway.
- ➢ Go AWS console navigates to internet gateway create internet gateway
- ➢ Attach internet gateway to recently created vpc.

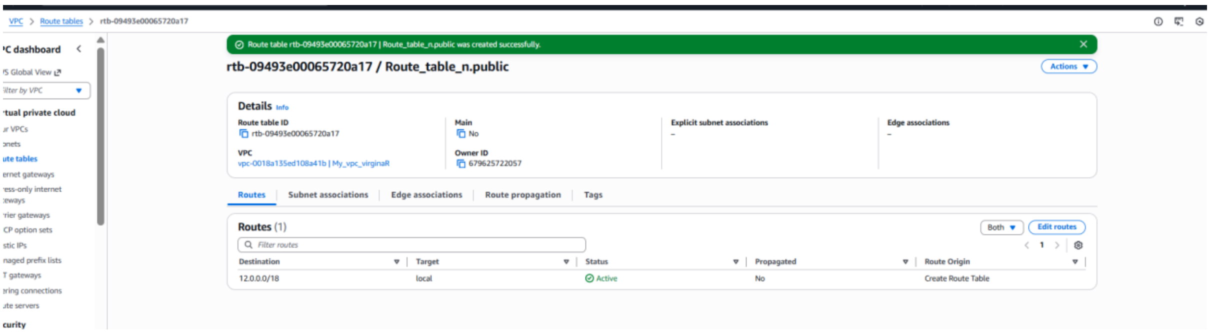


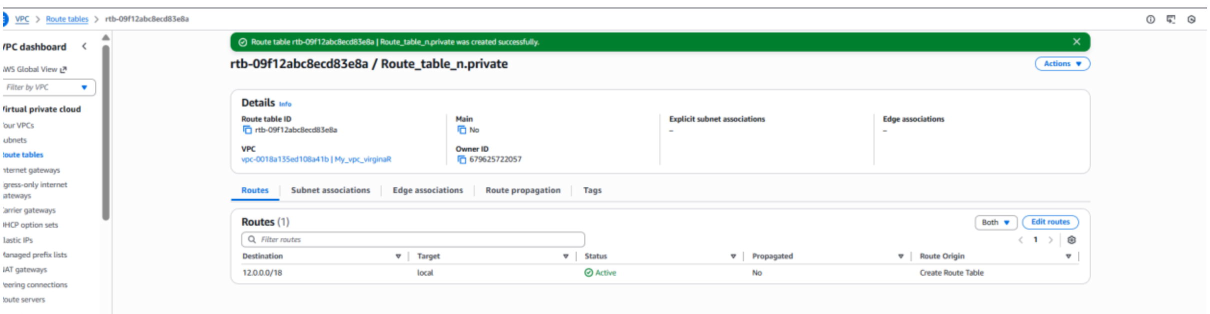


4. Create one public route table (RT) and one private route table.

Steps:

- ➢ Go to AWS console < search vpc < navigate to route table
- ➢ Create route table with recently created vpc and name it as public
- ➢ Create one more route table with recently created vpc and name it as priovate

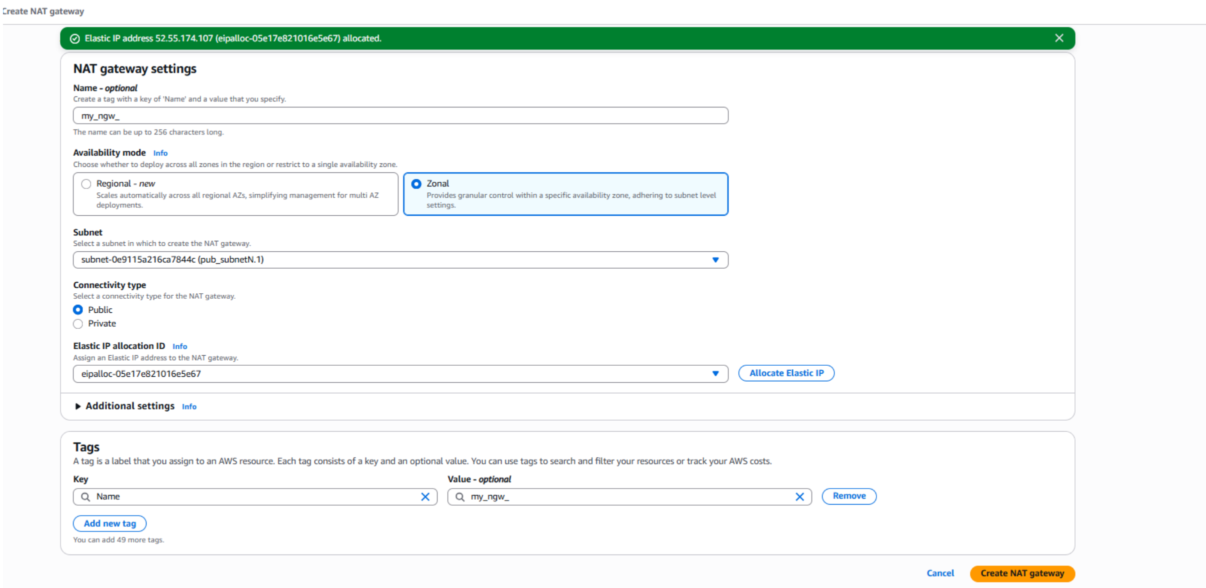




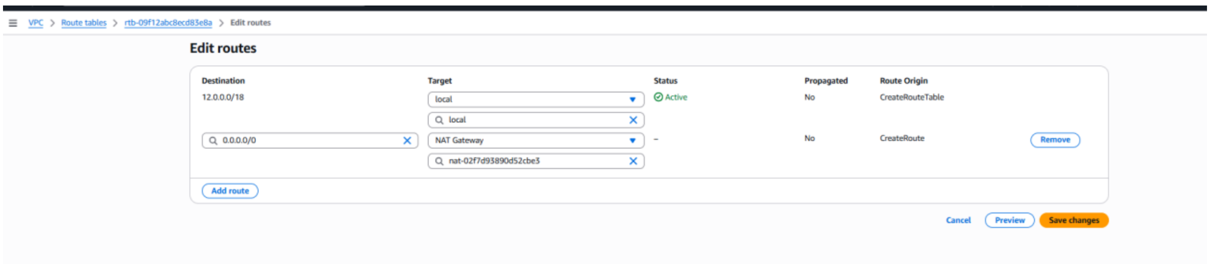
5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

Steps:

- Go to Natgateway < go to create natgateway
- Add public subnet while creating nat gateway
- Add nat gatway in private rote table



Added natgateway in route table but while created nat gateway public subnet is attached.



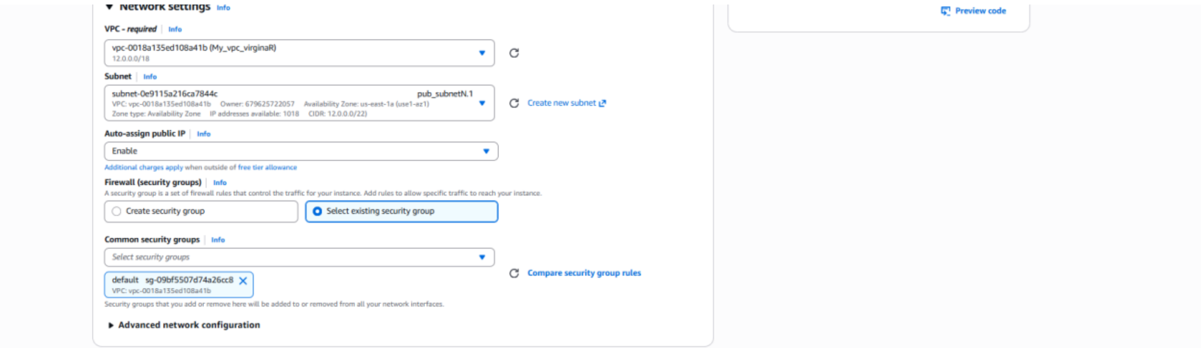
6. Create two instances, one in the public subnet and one in the private subnet.

7. Deploy Apache server on both EC2 instances with a sample index.html file.

Steps:

- Open AWS console launch an instance with public subnet
- While launching instance select recently created subnet and public subnet
- Add bash script in user data to launch hpptd with simple index file
- Launch another ec2 with private subnet and vpc which created

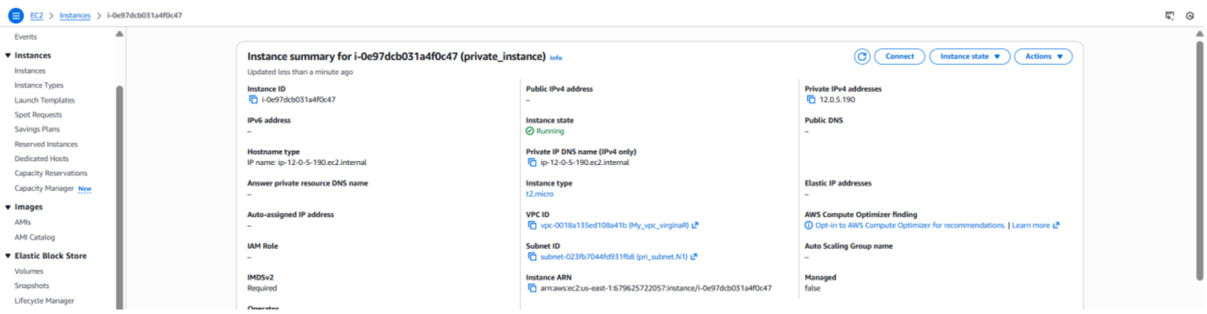
Launching instance with my the vpc which I have created and selected public subnet



Added bash script in user data while launching an instance

Deployed httpd on web.

Launched private subnet with my existing vpc which I have launched recently.

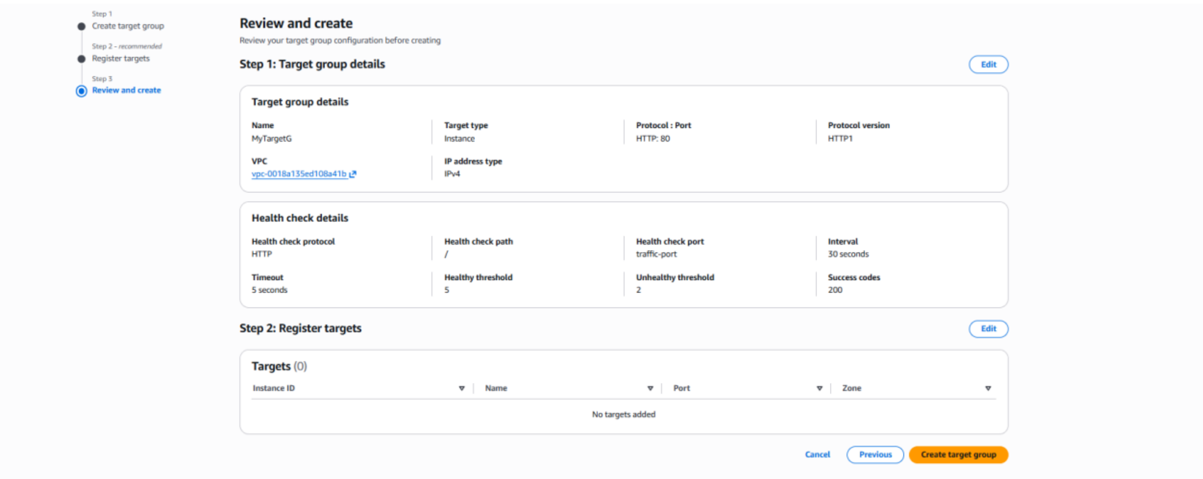


8. Create one application load balancer and attach it to both EC2 instance

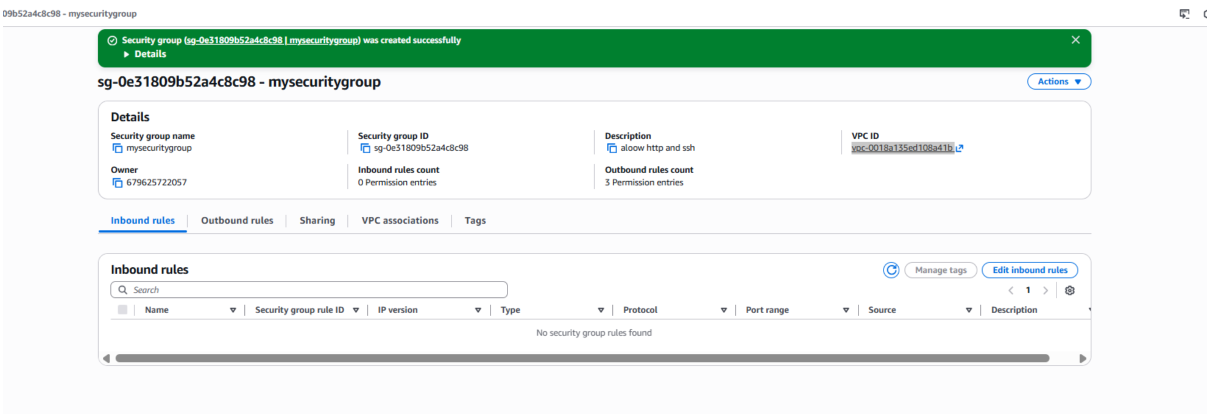
Steps:

- ➢ Create a target group go to ec2 console and select target group in left bottom
- ➢ While creating target group select vpc
- ➢ Create a load application balancer go to load balancer
- ➢ Select security group and select vpc and two public subnets
- ➢ Then attach that newly created security group while creating load balancer

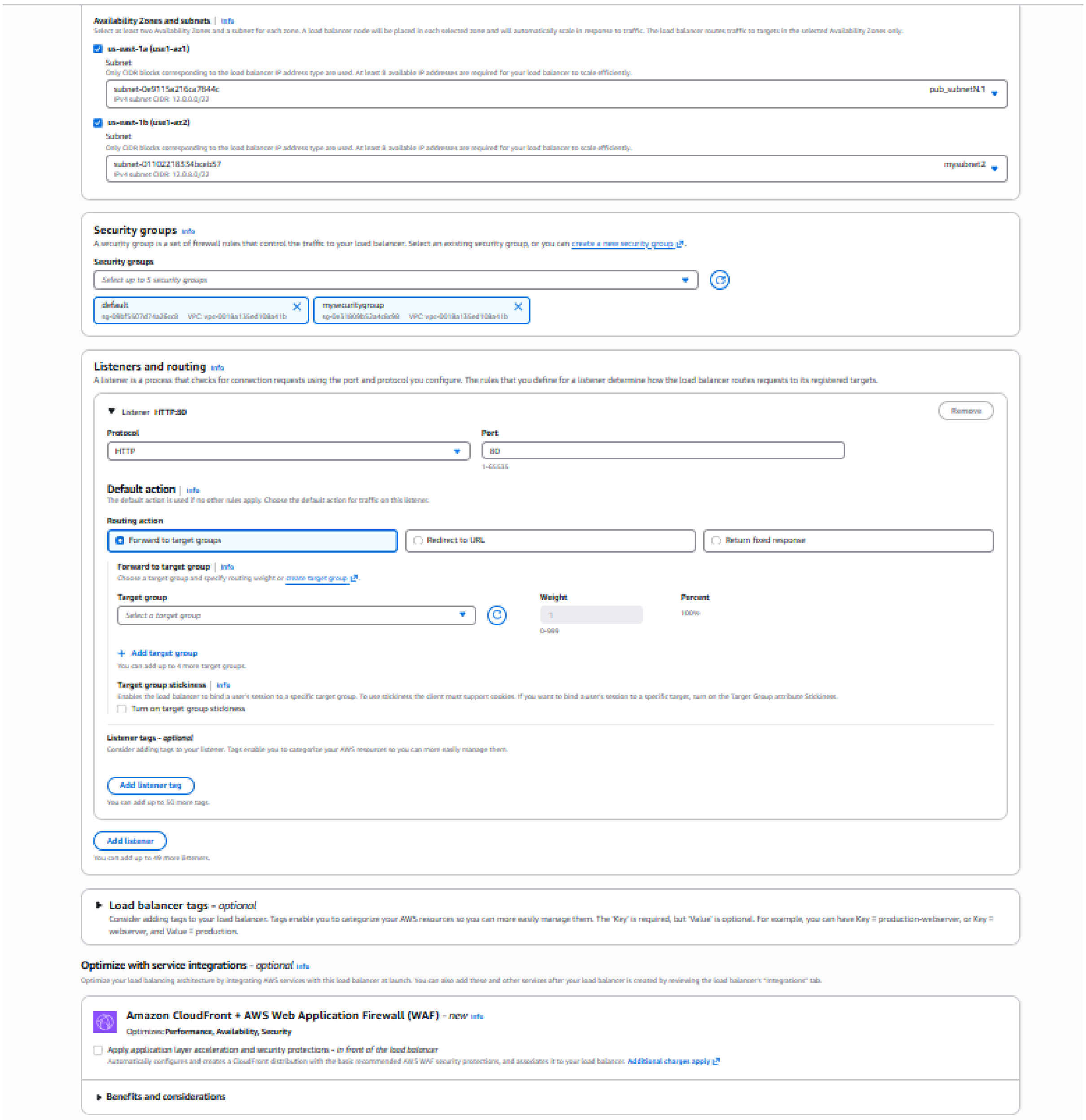
Created target group



Created security group

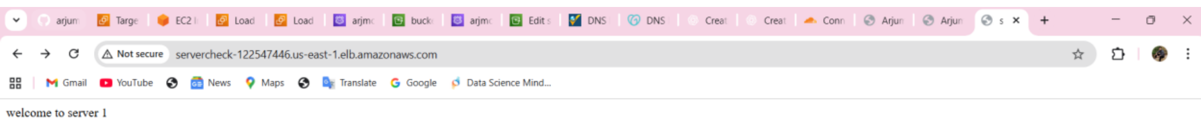


Created load balancer

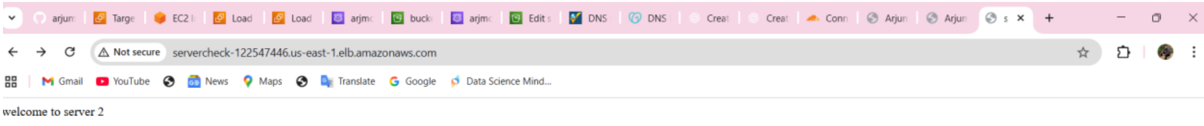


Created load balancer

Load balancer working fine



When I refresh it is giving second load balancer



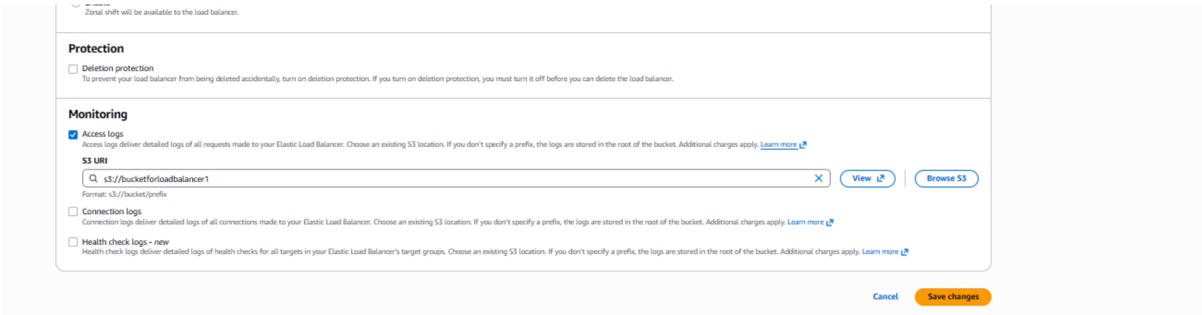
9. Store application load balancer logs in S3.

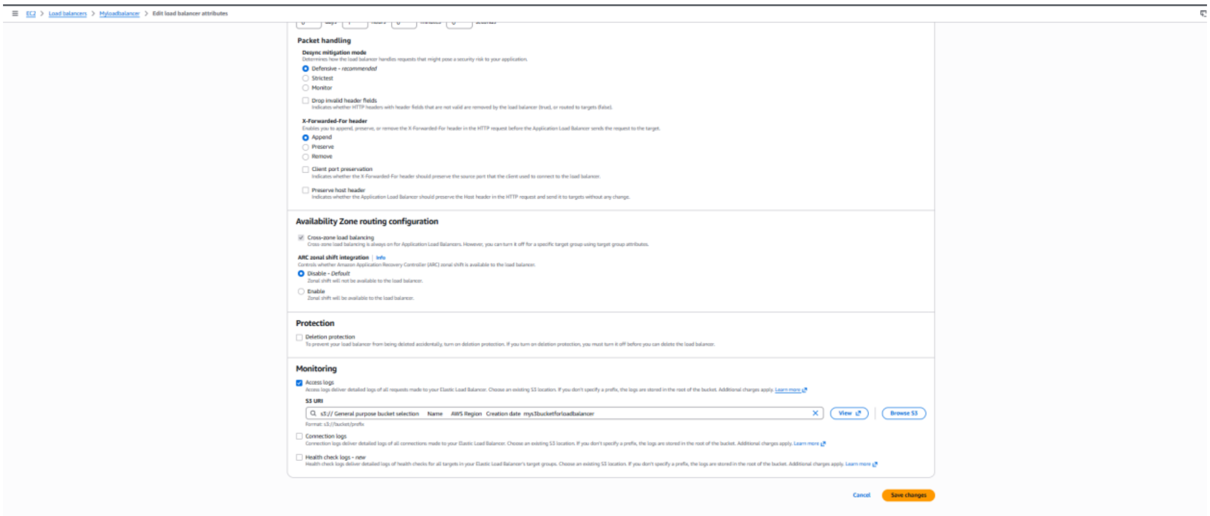
Steps:

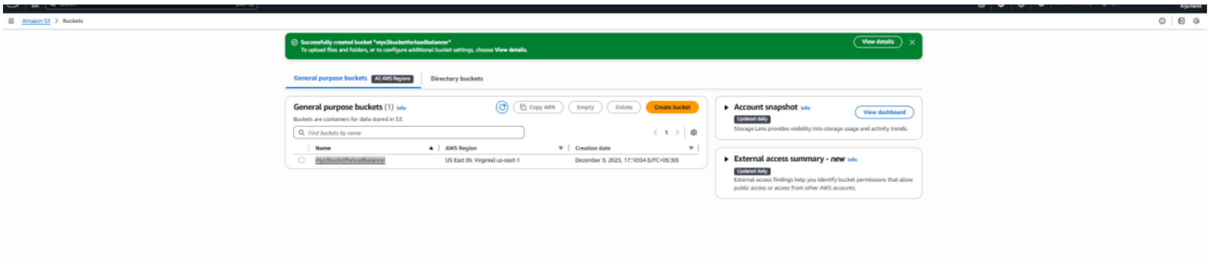
- Create s3 bucket and enable versioning
- Go to load balancer and go to attributes tab then click to edit and then enable access log in monitoring then add bucket name

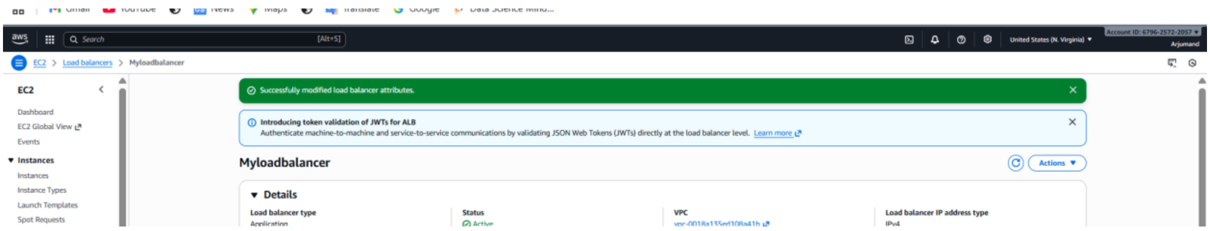
Bucket name is added in the below screen shot

Bucket is created in s3 north verginia region







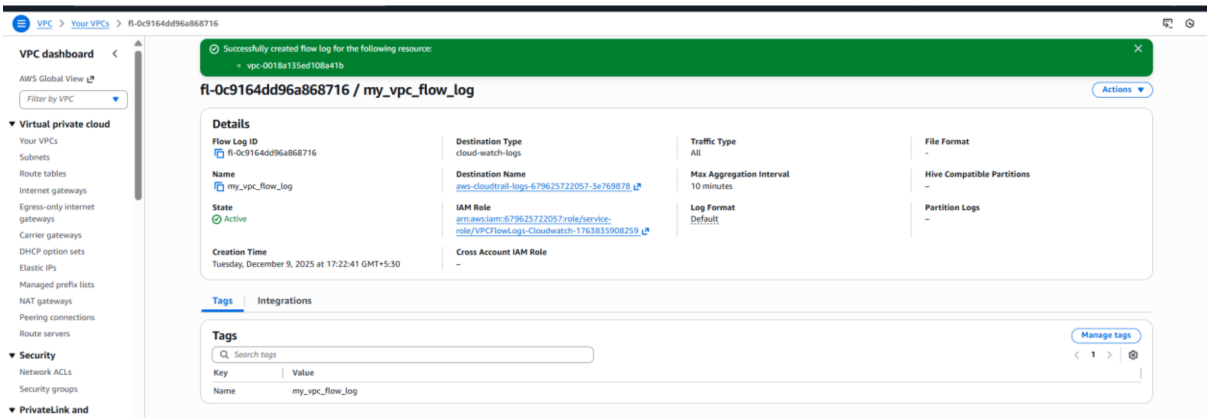


10. Store the VPC flow logs in a CloudWatch log group.

Steps:

- VPC → Your VPC → Flow Logs → Create Flow Log, choose Filter = ALL.
- Set Destination = CloudWatch Logs and create/select log group /vpc/flowlogs.
- Allow AWS to create IAM role → Create Flow Log → logs start flowing to CloudWatch.

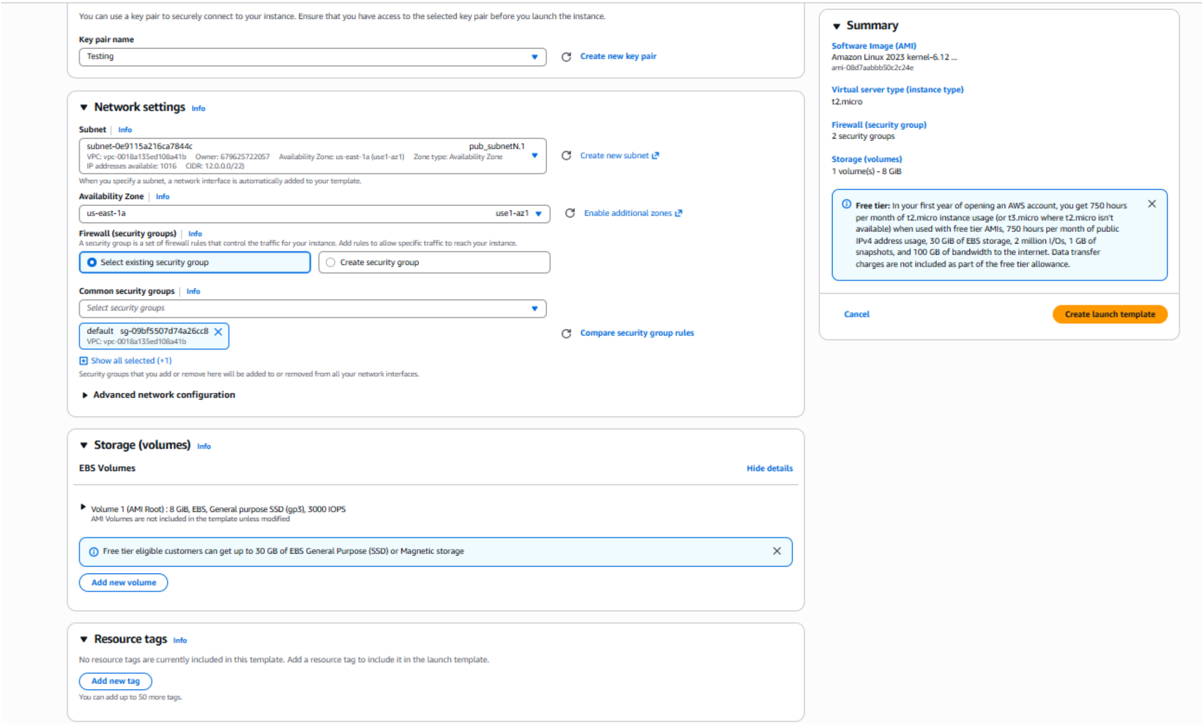
Created vpc flow log using cloudwatch



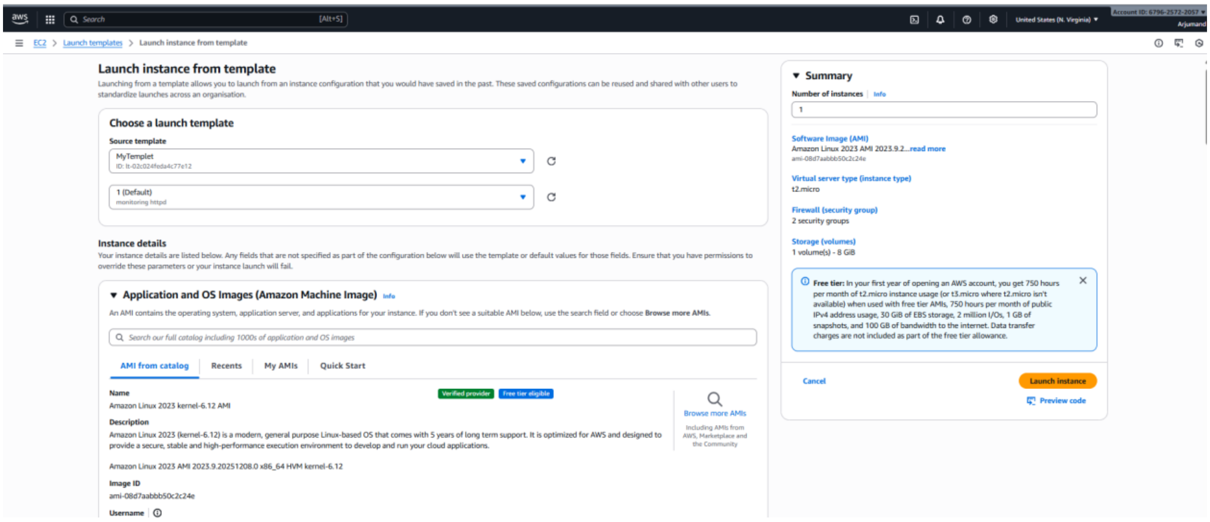
11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

Steps:

- Launch an EC2 instance (public subnet) and install Apache using sudo yum install httpd -y and start the service.
- Create a custom script apache-monitor.sh that checks Apache status and sends a custom metric to CloudWatch.
- Add a cron job (crontab -e) to run the script every 1 minute and continuously push the ApacheStatus metric.
- In CloudWatch → Metrics → Custom namespace, verify ApacheStatus metric is being received.
- Create a CloudWatch Dashboard and add a widget for EC2 CPUUtilization metric.
- Add a second widget and select your custom metric (ApacheMonitoring → ApacheStatus).
- Save the dashboard; now CPU usage and Apache service health are monitored in real time.



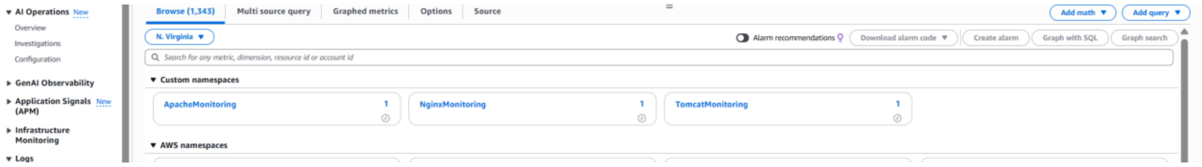
Creating instance with launch templet



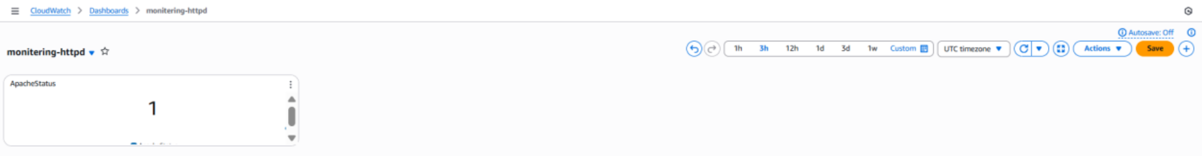
Created a bash script to create a matrics for httpd

```
#!/bin/bash
if systemctl is-active --quiet httpd; then STATUS=1; else STATUS=0; fi

aws cloudwatch put-metric-data \
  --metric-name ApacheStatus \
  --namespace "ApacheMonitoring" \
  --value $STATUS \
  --dimensions "Service=Apache"

~
~
~
~
~
```

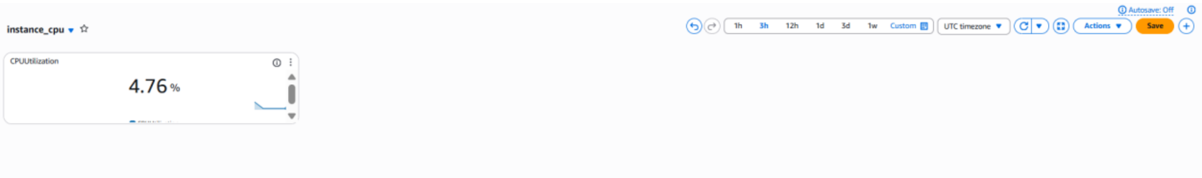
It shown in the image I successfully created matrics



Created a dashboard to monitor httpd



Created dashboard for ec2 monitoring



12. If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

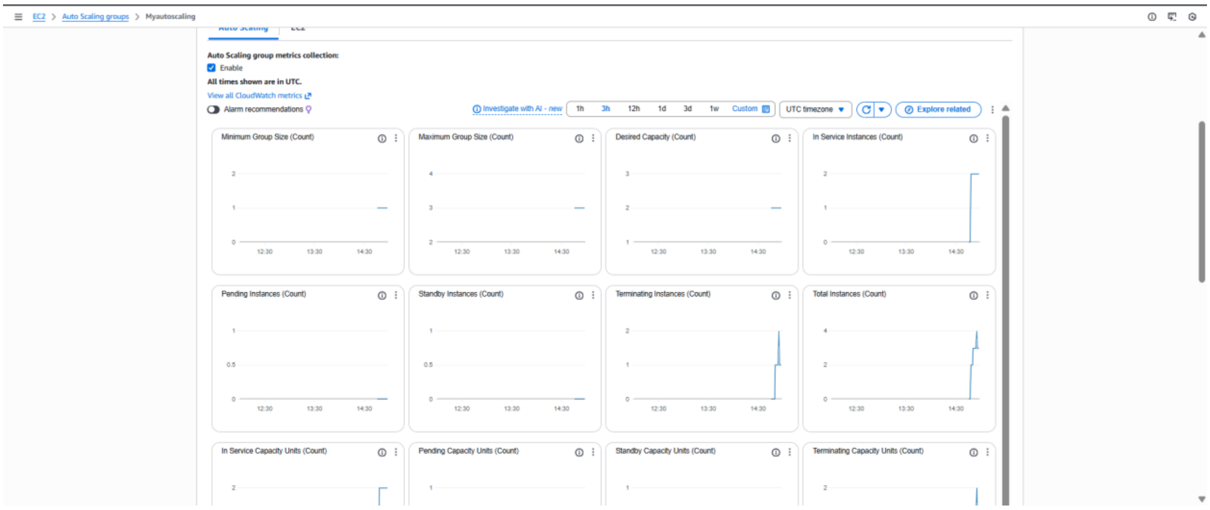
Steps:

- Create a **Launch Template** with Apache installation in *user data* so every new instance auto-configures.
- Create an **Auto Scaling Group (ASG)** using that launch template and select your public/private subnets.
- Attach the ASG to your **Application Load Balancer target group** so new instances register automatically.
- Set ASG values → **Desired = 1, Min = 1, Max = 4**.
- Add a **Target Tracking Scaling Policy** → Choose **CPUUtilization** and set **Target = 70%**.
- CloudWatch monitors CPU; when utilization exceeds 70%, ASG automatically launches a new EC2 instance.
- ALB forwards traffic to the new instance, and when CPU drops below threshold, ASG can scale in.

Created ASG with existing load balancer

Provided load

ASG is monitoring instsnace



Instance got triggered when load is increased

My newly created ASG instance is here

The newest running instance:
i-06101c4d40755659a → Public IP: 18.215.124.224
Status: **Initializing → Running**

👉 This is the **Auto-Scaled instance** launched automatically when CPU exceeded 70%.

2. My old EC2 instances (Public & Private) are also visible

- **i-064ae6b39ca47335** → Running → Public instance
- **i-0e97dcb3041fa0c47** → Stopped → Your private instance
- **i-0354823df09fd605** → Stopped → Another test instance
- **Terminated** old test instances.

3. ASG terminated an old instance

Example:
`i-015d36d8589893c85` → Terminated
This is NORMAL — ASG sometimes replaces old instances if:

- Health check failed
- Instance not responding
- Scaling activity required replacement